

RELATÓRIO CONFIDENCIAL · SEGURANÇA DE APLICAÇÃO

Relatório de Auditoria de Segurança — OrderFlow

Revisão dirigida a 5 categorias de falhas comuns em APIs próprias com frontend SPA

Data	14 de setembro de 2026
Repositório	OrderFlow (branch feature/implementacao-autoMapper)
Escopo	Backend ASP.NET Core (OrderFlow.Api / .Application / .Domain / .Infrastructure) e frontend React em client/
Fora de escopo	Não há pipelines de CI/CD (.github/workflows vazio) nem docker-compose/Helm/Terraform no repositório

NOTA METODOLÓGICA — MAPEAMENTO DE CATEGORIAS PARA A STACK DETECTADA

Stack: API própria em C# / ASP.NET Core 9 (Clean Architecture), EF Core 9 + Npgsql (PostgreSQL), autenticação JWT Bearer própria (sem provedor externo), FluentValidation, AutoMapper. Frontend em React 19 + TypeScript + Vite, roteamento client-side com react-router-dom.

Cat. 1 (tranca de banco): não é Supabase/RLS — mapeada para o único mecanismo de isolamento existente na stack: o atributo `[Authorize]` do ASP.NET Core em cada endpoint. Não há RLS, middleware de tenant nem filtro por `user_id`, pois o domínio (Cliente, Produto, Pedido) não modela propriedade por usuário.

Cat. 2: comparado cada gate de UI do React (`RequireAuth.tsx`) com os atributos de autorização dos Controllers.

Cat. 3 (IDOR): percorridos, sem amostragem, os 4 controllers e todos os handlers que recebem `Guid id`.

Cat. 4: revisados `appsettings*.json`, `Dockerfile`, `OrderFlow.Api.http`, `client/.env(example)` e o histórico git desses arquivos.

Cat. 5 (XSS): varredura em todo `client/src` por sinks (`dangerouslySetInnerHTML`, `innerHTML`, `eval`) e no backend por renderização de HTML/e-mail com entrada do usuário.

Resumo executivo

8ACHADOS
TOTAIS**3**

CRÍTICA

2

ALTA

2

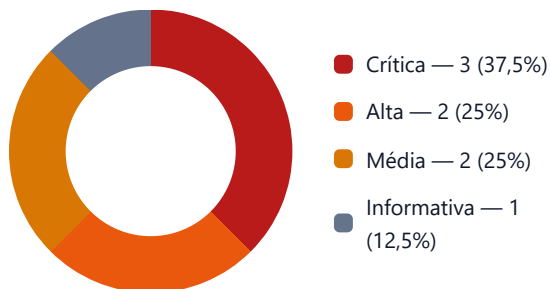
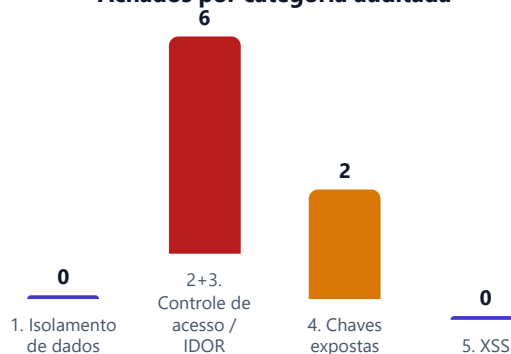
MÉDIA

0

BAIXA

1

INFORMATIVA

Achados por severidade**Achados por categoria auditada**

Leitura do resultado: as categorias 1 (isolamento por tenant/dono) e 5 (XSS) não geraram achados — a primeira porque a stack não modela propriedade de registros por usuário (nota abaixo), e a segunda porque o frontend não usa nenhum sink perigoso. O risco concentra-se quase inteiramente na categoria de controle de acesso: 3 dos 4 controllers expõem dados via GET sem qualquer autenticação.

Pontos fortes e pontos fracos

✓ Pontos fortes (verificados no código)

Hashing de senha robusto

`src/OrderFlow.Infrastructure/Security/SenhaHasher.cs:12-36`

PBKDF2-HMAC-SHA256, 100.000 iterações, salt aleatório de 16 bytes por usuário, e comparação em tempo constante (`CryptographicOperations.FixedTimeEquals`) para evitar timing attack.

Segredo JWT validado na inicialização (fail-fast)

`src/OrderFlow.Api/Program.cs:70-78`

`.Validate(s => s.Chave.Length >= 32).ValidateOnStart()` impede a API de subir com `Jwt.Chave` vazia ou fraca — em Produção (sem `appsettings.Development.json`) o processo falha ao iniciar em vez de assinar tokens com segredo inseguro.

Refresh token com rotação, hash e revogação

`src/OrderFlow.Application/Services/AuthService.cs:41-68` e `src/OrderFlow.Infrastructure/Security/TokenService.cs:45-52`

Apenas o hash SHA-256 do refresh token é persistido (nunca o valor em claro); a cada renovação o token antigo é revogado e substituído. Access token com TTL curto (15 min).

Rate limiting em login + limite global

`src/OrderFlow.Api/Program.cs:118-141`

Política "login" limita a 5 tentativas/min por IP; limitador global de 100 req/min por IP cobre o restante da API.

Rotas de criação (POST) corretamente protegidas

`ClientesController.cs:33`, `ProdutosController.cs:29`, `PedidosController.cs:50`

Todas as ações de escrita (criar cliente, criar produto, criar pedido) exigem `[Authorize]`. O problema identificado nesta auditoria está isolado às rotas de leitura (GET).

Consultas 100% parametrizadas via EF Core

`src/OrderFlow.Infrastructure/Repositories/*.cs`

Nenhum uso de `FromSqlRaw`/`FromSqlInterpolated`/`ExecuteSqlRaw` em todo o repositório — todas as consultas (incluindo filtros/paginação/ordenação dinâmica de `Pedidos`) usam LINQ parametrizado. Sem superfície de SQL Injection.

X Pontos fracos (riscos centrais)

Nenhuma política de autorização "fail-closed"

`src/OrderFlow.Api/Program.cs:95`

`builder.Services.AddAuthorization()` é registrado sem `FallbackPolicy`. Isso significa que qualquer endpoint novo nasce público por padrão, a menos que alguém lembre de adicionar `[Authorize]` manualmente — foi exatamente isso que aconteceu nos 5 endpoints GET.

Endpoints de leitura sem autenticação (achado central desta auditoria)

`ClientesController`, `ProdutosController`, `PedidosController`

Todas as rotas GET (listar e buscar por id) dos três controllers de negócio estão acessíveis anonimamente, expondo todos os clientes (nome+e-mail), produtos e pedidos. Detalhado na seção de achados.

Credenciais de banco de dados hardcoded

`src/OrderFlow.Api/appsettings.json:3`

Connection string com usuário/senha em texto plano commitados, sem validação de startup equivalente à do JWT rejeitando placeholders/secrets fracos.

Chave de assinatura JWT de desenvolvimento commitada

`src/OrderFlow.Api/appsettings.Development.json:3`

Segredo estático versionado no git; qualquer pessoa com acesso ao repositório pode forjar tokens válidos contra qualquer ambiente que rode com esse segredo.

Frontend sem sinks de XSS`client/src/**/*.tsx`

Nenhuma ocorrência de `dangerouslySetInnerHTML`, `innerHTML`, `eval` ou `new Function`. Toda a renderização de dados do servidor (nomes de cliente/produto, e-mails) passa pelo escurecimento automático do JSX.

Tokens mantidos só em memória no browser`client/src/auth/AuthContext.tsx:53-81`

`accessToken`/`refreshToken` vivem em `useRef`/`useState`, nunca em `localStorage`/`sessionStorage` — reduz o impacto de um eventual XSS futuro (o app documenta essa decisão na própria UI de depuração).

Middleware de erro não vaza stack trace em produção`src/OrderFlow.Api/Middleware/ExceptionMiddleware.cs:35-60`

Erros 5xx só retornam "Ocorreu um erro inesperado" fora de Development; detalhes completos vão apenas para o log do servidor com Traceld.

Achados detalhados por categoria

Categoria 1 — Isolamento de dados (tranca de banco / dono do registro)

Resultado: sem achados — categoria não aplicável na forma clássica. O domínio (Cliente, Produto, Pedido — ver `src/OrderFlow.Domain/Entities/`) não possui campo de propriedade por usuário nem conceito de organização/tenant; é um sistema de back-office de dado único, não uma SaaS multi-inquilino. Não há RLS (Postgres puro via EF Core) nem middleware de tenant a auditar. O mecanismo de isolamento real da aplicação é a própria autenticação — cuja ausência em rotas de leitura está registrada na Categoria 2+3 abaixo.

Categoria 2 — Permissão definida no navegador

Resultado: sem achados — categoria não aplicável. O frontend não possui nenhuma UI condicionada a papel/role (não há `isAdmin/canEdit` em nenhum componente); a `claim role` do JWT é apenas exibida no painel de depuração (`SessionPanel.tsx:48-49`), nunca usada para esconder/mostrar ações. Não existindo gate de UI por papel, não há gate correspondente a validar no backend nesta categoria.

Categoria 3 — IDOR / Controle de acesso quebrado nas rotas de leitura

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
CRÍTICA	<code>ClientesController.cs:23-29</code>	GET <code>/api/v1/clientes</code> (<code>ObterTodos</code>) não possui <code>[Authorize]</code>. Qualquer requisição anônima lista nome e e-mail de todos os clientes cadastrados. <pre>[HttpGet] public async Task<IActionResult> ObterTodos() { var clientes = await _clienteRepository.ObterTodosAsync(); return Ok(clientes); }</pre>
CRÍTICA	<code>ClientesController.cs:45-57</code>	GET <code>/api/v1/clientes/{id}</code> (<code>ObterPorId</code>) idem: sem <code>[Authorize]</code>. Combinado com o achado acima, os IDs (GUID) são trivialmente descobertos via a própria listagem, tornando o "segredo" do GUID irrelevante. <pre>[HttpGet("{id}")] public async Task<IActionResult> ObterPorId(Guid id) { var cliente = await _clienteRepository.ObterPorIdAsync(id); ... return Ok(cliente); }</pre>
CRÍTICA	<code>ProdutosController.cs:21-26</code>	GET <code>/api/v1/produtos</code> (<code>ObterTodos</code>) sem <code>[Authorize]</code>. Expõe catálogo completo, preços e estoque a qualquer visitante anônimo. <pre>[HttpGet] public async Task<IActionResult> ObterTodos() { var produtos = await _produtoRepository.ObterTodosAsync(); return Ok(produtos); }</pre>

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
ALTA	PedidosController.cs:34-47	GET /api/v1/pedidos (ObterTodos, paginado) sem [Authorize]. Expõe todos os pedidos — cliente vinculado, itens, valores — a qualquer chamador anônimo. Severidade Alta (e não Crítica) apenas porque os dados aqui derivam de Cliente/Produto já expostos pelos achados acima.
ALTA	PedidosController.cs:24-32	GET /api/v1/pedidos/{id} (ObterPorId) sem [Authorize] — leitura de qualquer pedido individual por ID sem nenhuma verificação. <pre>[HttpGet("{id}")] public async Task<IActionResult> ObterPorId(Guid id) { var response = await _pedidoService.ObterPorIdAsync(id); ... }</pre>
INFORMATIVA	TokenService.cs:28 / (ausência global)	O JWT carrega uma claim ClaimTypes.Role = usuario.Papel, mas nenhum Controller do projeto usa [Authorize(Roles = ...)] ou qualquer policy baseada em papel. Não é explorável por si só (nenhuma rota hoje depende disso), mas indica que, se uma rota administrativa for adicionada no futuro, não há padrão de RBAC pronto para protegê-la — risco de repetir o mesmo erro dos achados acima.

Categoria 4 — Chaves e segredos expostos (hardcode)

SEVERIDADE	ARQUIVO:LINHA	DESCRIÇÃO
MÉDIA	appsettings.json:3	Connection string com credenciais reais em texto plano, no arquivo base carregado em qualquer ambiente que não sobrescreva a chave via variável de ambiente. <pre>"DefaultConnection": "Host=localhost;Port=5432;Database=orderflow;Username=postgres;Password=po stgres"</pre> Ao contrário de Jwt:Chave, não há .Validate() rejeitando uma connection string placeholder/vazia — o startup não é "fail-closed" aqui. Condição de explorabilidade: só é um risco real se esse host/senha padrão permanecerem acessíveis fora do ambiente local do desenvolvedor (ex.: alguém reaproveita o valor padrão em Produção).
MÉDIA	appsettings.Development.json:3	Segredo de assinatura JWT commitado em texto plano (confirmado também no histórico do git — commit 65e9431 e anteriores). <pre>"Chave": "OrderFlow-Development-Secret-Key-Change-Before-Production-2026"</pre> O nome do valor já avisa "Change-Before-Production"; como o Dockerfile não define ASPNETCORE_ENVIRONMENT, o container roda em Production por padrão e este arquivo não é carregado nele (mitigação real). O risco fica restrito a quem rodar a API localmente com ASPNETCORE_ENVIRONMENT=Development apontando para um Postgres/rede compartilhados.

Categoria 5 — Inputs sem tratamento (XSS)

Resultado: sem achados. Varredura completa de client/src (todos os .tsx/.ts) não encontrou nenhuma ocorrência de dangerouslySetInnerHTML, innerHTML, eval, new Function ou URLs controladas por usuário em href/src. Todo dado

vindo da API (nomes de cliente/produto, e-mails, valores) é renderizado como texto via JSX, que escapa automaticamente. Não há sistema de e-mail nem renderização de templates HTML no backend, então a vertente server-side desta categoria não se aplica a este projeto — por isso também não há (nem seria necessária) biblioteca de sanitização como DOMPurify no `package.json` do client.

Recomendações priorizadas

P1 — Imediato

Adicionar [Authorize] às 5 rotas GET afetadas

ClientesController (ObterTodos, ObterPorId), ProdutosController (ObterTodos), PedidosController (ObterTodos, ObterPorId).

Trocar o modelo de autorização para "fail-closed"

Definir `FallbackPolicy = new AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build()` em `AddAuthorization (Program.cs:95)`, e usar `[AllowAnonymous]` apenas onde já é usado hoje (AuthController). Isso garante que qualquer endpoint futuro nasça protegido por padrão.

P2 — Próximo ciclo

Remover credenciais reais de appsettings.json

Mover a connection string para variável de ambiente / dotnet user-secrets / cofre de segredos; deixar no repositório apenas um placeholder óbvio (ex.: "SUBSTITUA_VIA_ENV"). Adicionar `.Validate()` no startup rejeitando esse placeholder, no mesmo padrão já usado para Jwt:Chave.

Rotacionar/parar de versionar a chave JWT de desenvolvimento

Gerar a chave localmente via dotnet user-secrets (fora do controle de versão) em vez de appsettings.Development.json.

P3 — Hardening / melhoria contínua

Decidir o papel do RBAC

Ou aplicar `[Authorize(Roles="Admin")]` explicitamente quando surgir uma rota administrativa, ou remover a claim de papel se ela não tiver uso planejado — deixar "pela metade" convida a repetir o achado central desta auditoria.

Teste de regressão para autorização

Um teste de integração que itera todos os endpoints via reflection/roteamento e falha se algum novo endpoint mutante ficar sem `[Authorize]` evita reincidência do achado P1.

Rate limiting no endpoint de registro

Hoje só `/auth/login` tem a policy dedicada "login"; `/auth/registrar` cai no limite global de 100 req/min por IP, mais permissivo para criação de contas em massa.

Issues para o GitHub

Texto pronto para copiar e colar na criação de issues. Achados triviais do mesmo tema foram agrupados para evitar spam.

--- ISSUE 1 ---

Título

[Segurança] Endpoints de leitura (GET) sem autenticação expõem todos os Clientes, Produtos e Pedidos

Labels sugeridas

security, crítica

Descrição do problema

Todas as rotas GET dos controllers de negócio (Clientes, Produtos, Pedidos) estão marcadas implicitamente como anônimas: nenhuma delas tem o atributo `[Authorize]`, ao contrário das rotas POST (criar), que o têm corretamente. Como `AddAuthorization()` em Program.cs não define uma FallbackPolicy, o ASP.NET Core trata qualquer endpoint sem `[Authorize]` como público por padrão. Isso é explorável por qualquer cliente HTTP, sem token, sem autenticação alguma.

Por que é explorável

GUIDs de Cliente/Produto/Pedido não são segredo – a própria listagem (`GET /api/v1/clientes`, `GET /api/v1/produtos`) devolve todos os IDs existentes. Um atacante não autenticado consegue: (1) listar todos os clientes com nome+e-mail; (2) listar todo o catálogo de produtos com preço/estoque; (3) listar e paginar todos os pedidos com valores e itens; (4) buscar qualquer registro individual por ID.

Evidência

- src/OrderFlow.Api/Controllers/ClientesController.cs:23-29 (ObterTodos) e :45-57 (ObterPorId)
...

```
[HttpGet]
public async Task<IActionResult> ObterTodos()
{
    var clientes = await _clienteRepository.ObterTodosAsync();
    return Ok(clientes);
}
...
```

- src/OrderFlow.Api/Controllers/ProdutosController.cs:21-26 (ObterTodos)
...

```
[HttpGet]
public async Task<IActionResult> ObterTodos()
{
    var produtos = await _produtoRepository.ObterTodosAsync();
    return Ok(produtos);
}
...
```

- src/OrderFlow.Api/Controllers/PedidosController.cs:34-47 (ObterTodos) e :24-32 (ObterPorId)
...

```
[HttpGet]
public async Task<IActionResult> ObterTodos([FromQuery] ConsultaPedidosDto parametros)
{
    var (itens, totalItens) = await _pedidoService.ObterPaginadoAsync(parametros);
    ...
}
...
```

Impacto

Vazamento em massa de dados de clientes (PII: nome + e-mail), catálogo comercial (preços/estoque) e histórico de pedidos (dados de negócio sensíveis) para qualquer parte não autenticada – sem necessidade de nenhuma feature flag ou configuração insegura adicional; a falha se manifesta em qualquer deploy padrão desta branch.

Sugestão de correção

1. Adicionar `[Microsoft.AspNetCore.Authorization.Authorize]` às 5 ações GET listadas acima.
2. Estruturalmente, trocar `builder.Services.AddAuthorization()` (Program.cs:95) por uma versão com `FallbackPolicy = new AuthorizationPolicyBuilder().RequireAuthenticatedUser().Build()`, mantendo `[AllowAnonymous]` apenas nas 4 rotas do AuthController que já o usam hoje. Isso torna o modelo "fail-closed" e evita a reincidência do problema em endpoints futuros.

Critérios de aceite

- [] `GET /api/v1/clientes` sem token retorna 401
- [] `GET /api/v1/clientes/{id}` sem token retorna 401
- [] `GET /api/v1/produtos` sem token retorna 401

- [] `GET /api/v1/pedidos` sem token retorna 401
- [] `GET /api/v1/pedidos/{id}` sem token retorna 401
- [] Os mesmos endpoints continuam retornando 200 com um JWT válido
- [] Existe teste de integração que falha caso um novo endpoint seja adicionado sem `[Authorize]` (ou sem estar coberto pela FallbackPolicy)
- [] Rotas `/auth/\*` continuam acessíveis anonimamente (login/registro/refresh/revogação)

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título

[Segurança] Segredos e credenciais padrão commitados nos arquivos de configuração

Labels sugeridas

security, média

Descrição do problema

Dois segredos estão em texto plano, versionados no git:

1. A connection string do PostgreSQL em `appsettings.json` (usuário e senha reais, embora sejam o padrão local `postgres`/`postgres`).
2. A chave de assinatura JWT de desenvolvimento em `appsettings.Development.json`.

Diferente da chave JWT (que tem `.Validate(s => s.Chave.Length >= 32).ValidateOnStart()` em Program.cs), a connection string não tem nenhuma validação de startup que rejeite um valor placeholder/inseguro – não há tratamento simétrico entre os dois segredos.

Por que é explorável

Qualquer pessoa com acesso de leitura ao repositório (ou ao histórico do git – confirmado presente desde os commits iniciais) tem as credenciais de banco e a chave de assinatura de tokens. Se qualquer um dos dois valores padrão sobreviver, sem substituição, em um ambiente alcançável pela rede (banco de dados exposto com a senha `postgres`, ou API rodando com `ASPNETCORE\_ENVIRONMENT=Development` fora do laptop do desenvolvedor), o atacante pode ler/escrever diretamente no banco ou forjar JWTs válidos para qualquer usuário.

Evidência

- src/OrderFlow.Api/appsettings.json:3

...

"DefaultConnection": "Host=localhost;Port=5432;Database=orderflow;Username=postgres;Password=postgres"

...

- src/OrderFlow.Api/appsettings.Development.json:3

...

"Chave": "OrderFlow-Development-Secret-Key-Change-Before-Production-2026"

...

Impacto

Comprometimento total do banco de dados (leitura/escrita/exclusão) ou falsificação de tokens de autenticação, condicionado a esses valores padrão permanecerem em uso fora do ambiente local isolado do desenvolvedor. Em produção real (container Docker), a chave JWT deste arquivo não é sequer carregada, pois o Dockerfile não define `ASPNETCORE\_ENVIRONMENT` (efeito é Production) – isso reduz, mas não elimina, o risco.

Sugestão de correção

1. Substituir os valores reais por placeholders óbvios (`"SUBSTITUA\_VIA\_VARIAVEL\_DE\_AMBIENTE"`) nos dois arquivos.
2. Configurar a connection string real via variável de ambiente (`ConnectionStrings\_\_DefaultConnection`) ou `dotnet user-secrets` em desenvolvimento, nunca em arquivo versionado.
3. Adicionar validação de startup (`.Validate(...).ValidateOnStart()`) rejeitando a connection string quando ausente/placeholder, no mesmo padrão já aplicado a `Jwt:Chave`.
4. Gerar a chave JWT de desenvolvimento localmente via `dotnet user-secrets set "Jwt:Chave" "..."` e remover o valor de `appsettings.Development.json`.

Critérios de aceite

- [] `appsettings.json` e `appsettings.Development.json` não contêm mais segredos reais
- [] A aplicação falha ao iniciar (fail-fast) se `ConnectionStrings:DefaultConnection` estiver ausente ou for o placeholder
- [] README/documentação de setup local explica como configurar os segredos via user-secrets/variáveis de ambiente
- [] Segredos antigos (chave JWT de dev) são considerados comprometidos e não reutilizados

--- FIM ISSUE 2 ---